Sometimes, you will need to access the service container directly for a service. This approach is known as the *service locator* pattern. That sounds like it's a good thing, being a design pattern and all, but it is generally considered an *anti-pattern* and should be avoided. However, sometimes you don't have a choice. You have already seen an example of this when you used a factory to register a service that takes another service as a dependency earlier in the chapter.

DEFINITION An anti-pattern is a commonly used solution to a recurring problem (a pattern), that is usually suboptimal in some way. This might be because the solution introduces new problems or because it simply moves the problem elsewhere.

The `IServiceProvider` service provides access to registered services. It has one method, `GetService`, which returns the specified service, or `null` if it is not found. In addition, there is an extension method, `GetRequiredService`, that throws an exception if the specified service is not found. The `IServiceProvider` is injected into the consumer, which then uses it to retrieve the services it needs.

Listing 7.17 Example usage of the service locator pattern

```
public class IndexModel : PageModel
{
    private readonly IServiceProvider _serviceProvider;          ❶

    public IndexModel(IServiceProvider serviceProvider) =>]
    ➥  _serviceProvider = serviceProvider;                      ❶

    public List<City> Cities { get; set; }                       ❶
    public async Task OnGetAsync()
    {
        var cityService =
        ➥ _serviceProvider.GetRequiredService<ICityService>();  ❶
        Cities = await cityService.GetAllAsync();
    }
}
```

❶ Inject the IServiceProvider into the class constructor.